

COSMIC CRAFT

AI COMIC STORY CREATOR USING GEMINI MODELS

CODING + OUTPUT DOCUMENTATION (Code First, Output Next)

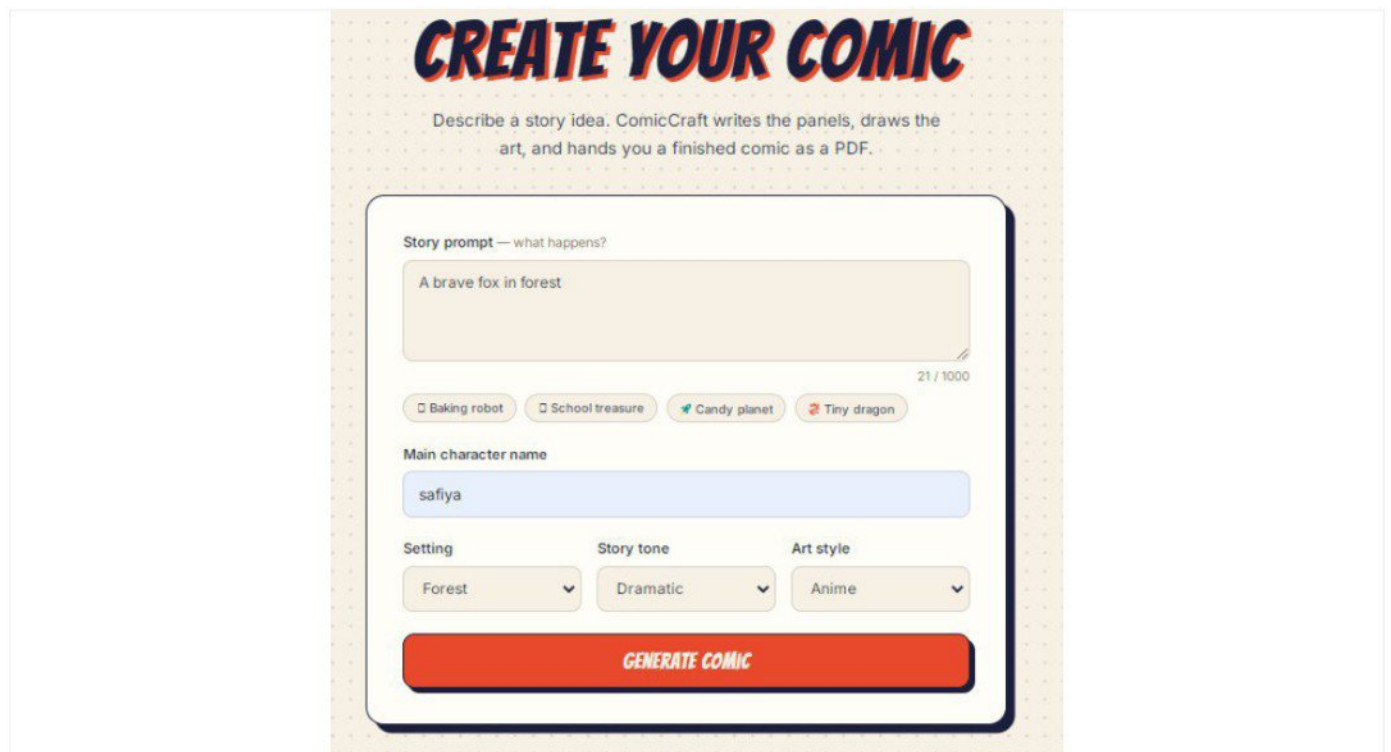
Documentation pattern: First shows core coding implementation for each module, then shows corresponding UI as OUTPUT.
Based on Gemini Flash, Gemini Pro and Stable Diffusion pipeline.

1. Project Structure Setup

Initial project structure creation as shown.

```
comiccraft/  
|-- app/  
| |-- main.py  
| |-- routes.py  
| |-- gemini_flash.py    # generate_outline()  
| |-- gemini_pro.py      # generate_story()  
| |-- image_generator.py # generate_image()  
| |-- layout_builder.py  # build_comic_layout()  
| +-- exporters.py       # save_pdf()  
|-- templates/  
| |-- index.html  
| |-- comic_preview.html  
| +-- export_success.html  
|-- static/  
| |-- panels/ # Generated panel images  
| |-- exports/ # Exported comic PDFs  
| +-- fonts/  
|-- .env # API keys  
+-- requirements.txt
```

OUTPUT:



The screenshot displays the 'CREATE YOUR COMIC' web application. The interface features a large, stylized title at the top, followed by a descriptive sentence: 'Describe a story idea. ComicCraft writes the panels, draws the art, and hands you a finished comic as a PDF.' Below this, there is a form with several input fields and buttons. The 'Story prompt' field contains the text 'A brave fox in forest'. Below the prompt field, there are four buttons: 'Baking robot', 'School treasure', 'Candy planet' (which is selected), and 'Tiny dragon'. The 'Main character name' field contains the text 'safiya'. At the bottom, there are three dropdown menus for 'Setting' (Forest), 'Story tone' (Dramatic), and 'Art style' (Anime). A large red button labeled 'GENERATE COMIC' is positioned at the bottom of the form.

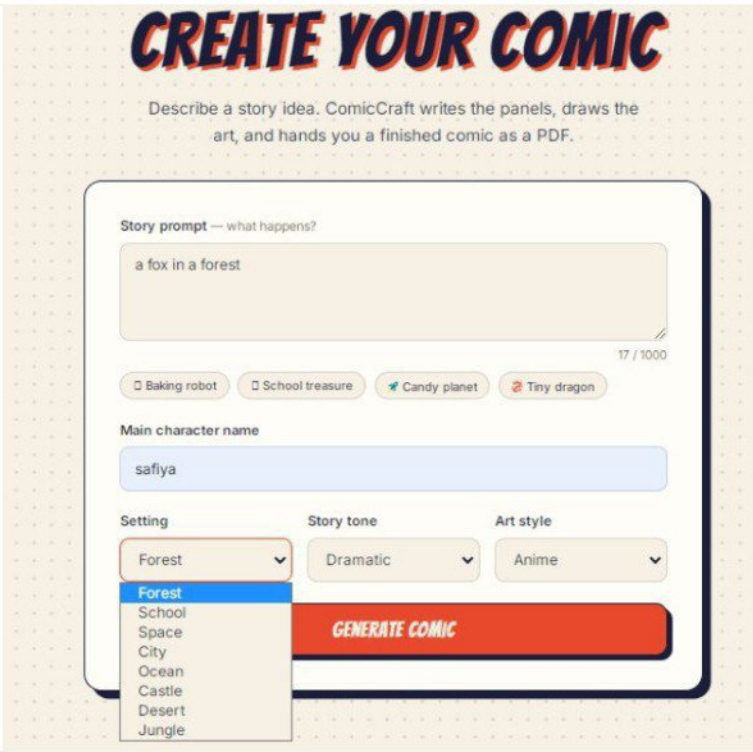
2. Generate Structured Comic Panel Outline - gemini_flash.py

Uses Gemini Flash to generate 5-panel structured outline.

```
import google.generativeai as genai
import json

def generate_outline(prompt, character, setting, tone, art_style):
    model = genai.GenerativeModel('models/gemini-1.5-flash')
    system_prompt = f"""
    You are a comic writer. Create 5-panel outline for:
    Story: {prompt}
    Character: {character}, Setting: {setting}
    Tone: {tone}, Art Style: {art_style}
    Return JSON list with: panel_number, title, scene_description, image_prompt
    """
    response = model.generate_content(system_prompt)
    outline = json.loads(response.text)
    return outline
```

OUTPUT:



The screenshot displays the 'CREATE YOUR COMIC' web interface. At the top, the title 'CREATE YOUR COMIC' is in a large, bold, blue font. Below it, a subtitle reads: 'Describe a story idea. ComicCraft writes the panels, draws the art, and hands you a finished comic as a PDF.' The main form is a rounded rectangle with a light blue border. It contains several input fields and buttons. The 'Story prompt' field is a large text area with the placeholder text 'a fox in a forest' and a character count '17 / 1000'. Below it are four buttons: 'Baking robot', 'School treasure', 'Candy planet' (which is highlighted with a green checkmark), and 'Tiny dragon'. The 'Main character name' field is a text input with the value 'safiya'. Below this are three dropdown menus: 'Setting' (with 'Forest' selected and a dropdown menu open showing options like Forest, School, Space, City, Ocean, Castle, Desert, and Jungle), 'Story tone' (with 'Dramatic' selected), and 'Art style' (with 'Anime' selected). A large red button labeled 'GENERATE COMIC' is positioned at the bottom right of the form.

3. Generate Comic Story Narration - gemini_pro.py

Gemini Pro expands outline into full story with captions and dialogues.

```
def generate_story(outline, character, tone):
    model = genai.GenerativeModel('models/gemini-1.5-pro')
    story_text = ""
    for panel in outline:
        prompt = f"""
        Panel {panel['panel_number']}: {panel['title']}
        Scene: {panel['scene_description']}
        Character: {character}, Tone: {tone}
        Write engaging narration + dialogues (Caption + Narration).
        """
        res = model.generate_content(prompt)
        story_text += res.text + "\n\n"
    return story_text
```

OUTPUT:

The screenshot displays the 'CREATE YOUR COMIC' web interface. At the top, the title 'CREATE YOUR COMIC' is in a large, bold, stylized font. Below it, a subtitle reads: 'Describe a story idea. ComicCraft writes the panels, draws the art, and hands you a finished comic as a PDF.' The main form is titled 'Story prompt — what happens?' and contains a text input field with the value 'a fox in a forest'. To the right of the input field is a character count '17 / 1000'. Below the input field are four buttons: 'Baking robot', 'School treasure', 'Candy planet' (which is selected and highlighted with a green checkmark), and 'Tiny dragon'. The form also has a 'Main character name' input field with the value 'safiya'. At the bottom, there are three dropdown menus: 'Setting' (set to 'Forest'), 'Story tone' (set to 'Dramatic' with a dropdown menu open showing options: 'Dramatic', 'Light-hearted', 'Funny', 'Mysterious', 'Adventurous', and 'Poetic'), and 'Art style' (set to 'Anime').

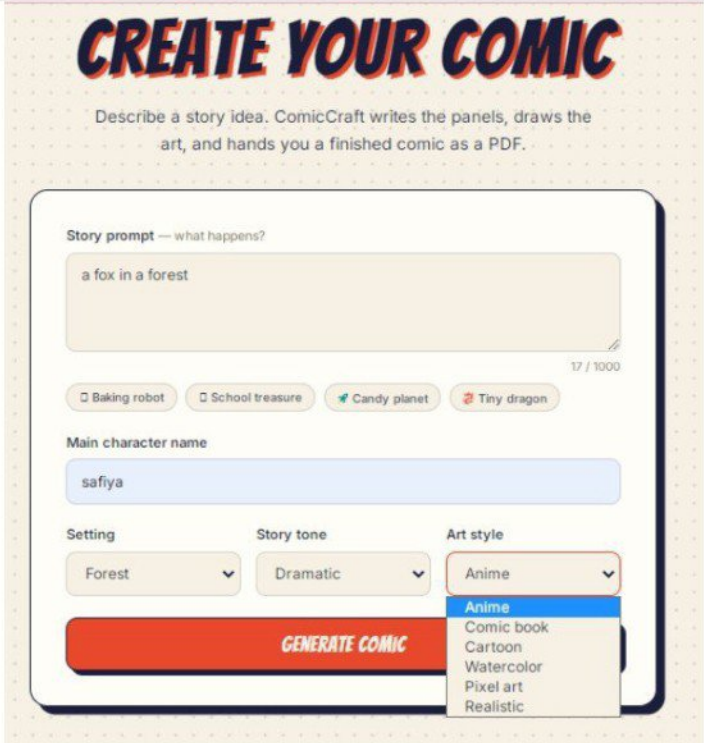
4. Generate Comic Illustrations - image_generator.py

Stable Diffusion generates comic-style images from prompts.

```
from diffusers import StableDiffusionPipeline
import torch, os
pipe = StableDiffusionPipeline.from_pretrained('runwayml/stable-diffusion-v1-5').to('cpu')

def generate_image(image_prompt, panel_number):
    safe_prompt = f"comic book style, {image_prompt}, vibrant colors"
    image = pipe(safe_prompt, num_inference_steps=30).images[0]
    filename = f"panel_{panel_number}_{image_prompt[:20].replace(' ', '_')}.png"
    path = os.path.join('static/panels', filename)
    image.save(path)
    return path
```

OUTPUT:



CREATE YOUR COMIC

Describe a story idea. ComicCraft writes the panels, draws the art, and hands you a finished comic as a PDF.

Story prompt — what happens?

a fox in a forest

17 / 1000

☐ Baking robot ☐ School treasure ☒ Candy planet ☐ Tiny dragon

Main character name

safiya

Setting: Forest

Story tone: Dramatic

Art style: Anime

GENERATE COMIC

Anime
Comic book
Cartoon
Watercolor
Pixel art
Realistic

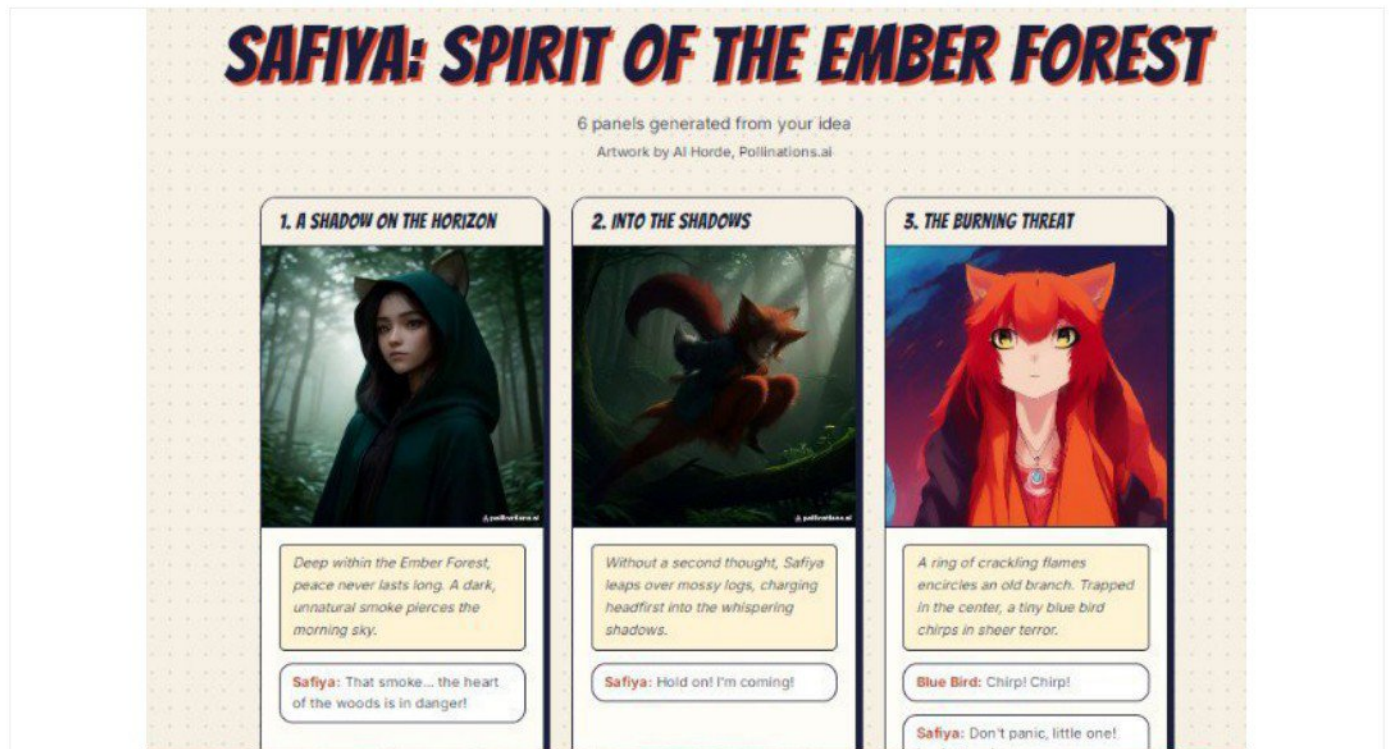
5. Organize Layout & Export PDF - layout_builder.py & exporters.py

Builds final panel layout and exports to PDF using FPDF.

```
def build_comic_layout(outline, images, story):
    layout = []
    panels_story = story.split("\n\n")
    for i, panel in enumerate(outline):
        layout.append({
            'number': panel['panel_number'],
            'title': panel['title'],
            'image': images[i],
            'scene': panel['scene_description'],
            'text': panels_story[i] if i < len(panels_story) else "",
            'image_prompt': panel['image_prompt']
        })
    return layout

from fpdf import FPDF
def save_pdf(layout, filename='comic.pdf'):
    pdf = FPDF()
    for panel in layout:
        pdf.add_page()
        pdf.set_font('Arial','B',16)
        pdf.cell(0,10, f"Panel {panel['number']}: {panel['title']}", ln=True)
        pdf.image(panel['image'], x=10, y=30, w=180)
        pdf.set_xy(10,150)
        pdf.set_font('Arial','',11)
        pdf.multi_cell(0,8, panel['text'])
    pdf.output(f"static/exports/{filename}")
    return f"static/exports/{filename}"
```

OUTPUT:



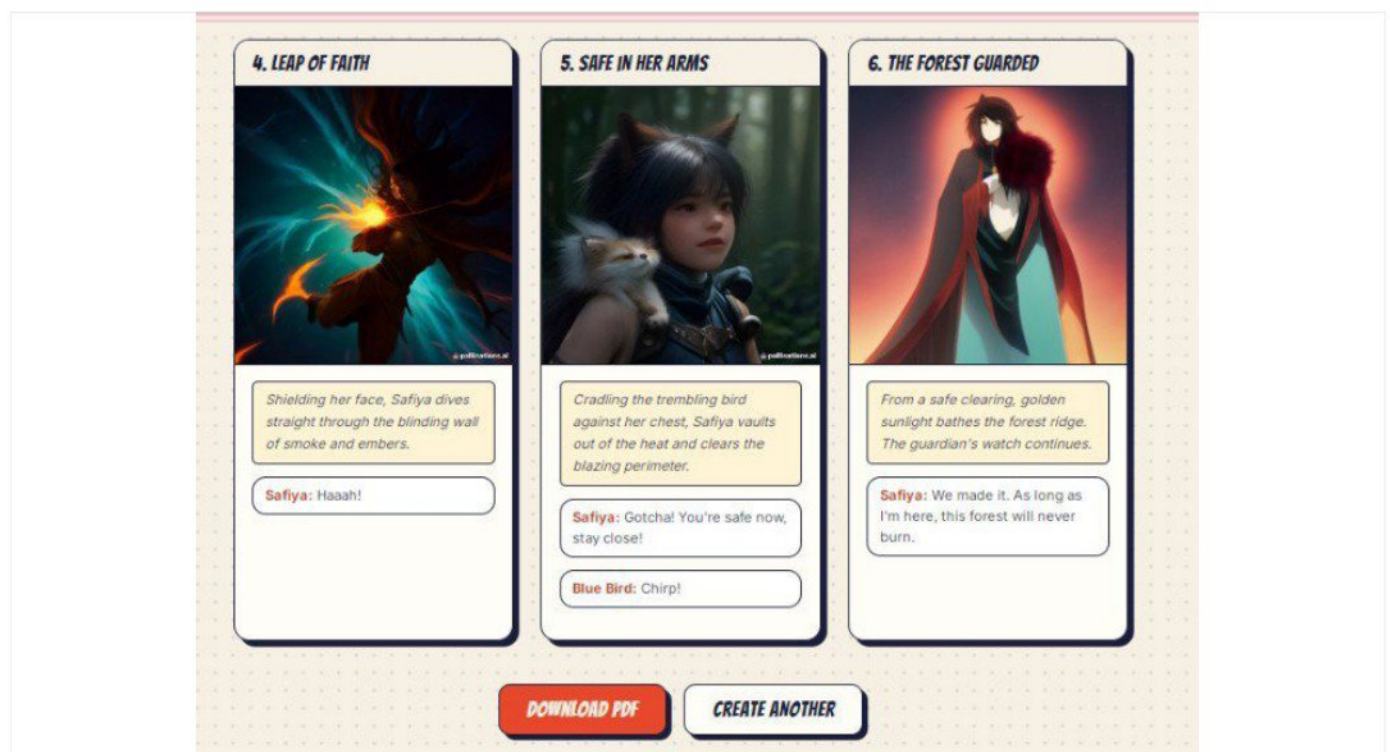
6. FastAPI Routing - routes.py

Handles form submission, calls all AI workflows and renders comic_preview.html.

```
from fastapi import APIRouter, Form, Request
from fastapi.templating import Jinja2Templates
router = APIRouter()
templates = Jinja2Templates(directory='templates')

@router.post('/generate')
async def generate_comic(request: Request, prompt: str = Form(...), character: str = Form(...), setting: str = Form(...), tone: str = Form(...), art_style: str = Form(...)):
    outline = generate_outline(prompt, character, setting, tone, art_style)
    story = generate_story(outline, character, tone)
    images = [generate_image(p['image_prompt'], p['panel_number']) for p in outline]
    layout = build_comic_layout(outline, images, story)
    pdf_path = save_pdf(layout)
    return templates.TemplateResponse('comic_preview.html', {'request': request, 'layout': layout, 'pdf_path': pdf_path})
```

OUTPUT:



7. Frontend UI Flow - Final Outputs

Homepage collects Story Prompt, Character, Setting, Tone, Art Style. Preview shows panel-wise comic with images, scene description, captions, narration. Export Success confirms PDF download.

CREATE YOUR COMIC

Describe a story idea. ComicCraft writes the panels, draws the art, and hands you a finished comic as a PDF.

Story prompt — what happens?

A brave fox in forest

21 / 1000

Baking robot

School treasure

Candy planet

Tiny dragon

Main character name

safiya

Setting

Story tone

Art style

Forest

Dramatic

Anime

GENERATE COMIC

CREATE YOUR COMIC

Describe a story idea. ComicCraft writes the panels, draws the art, and hands you a finished comic as a PDF.

Story prompt — what happens?

a fox in a forest

17 / 1000

Baking robot

School treasure

Candy planet

Tiny dragon

Main character name

safiya

Setting

Story tone

Art style

Forest

Dramatic

Anime

GENERATE COMIC

Conclusion

ComicCraft successfully demonstrates AI-driven comic generation. Combining Gemini Flash for outline, Gemini Pro for narration, Stable Diffusion for illustrations, it transforms simple prompts into complete visually engaging comics.

Workflow: User Input -> Outline (Flash) -> Story (Pro) -> Images (SD) -> Layout Builder -> PDF Export (FPDF) -> Preview -> Download.